

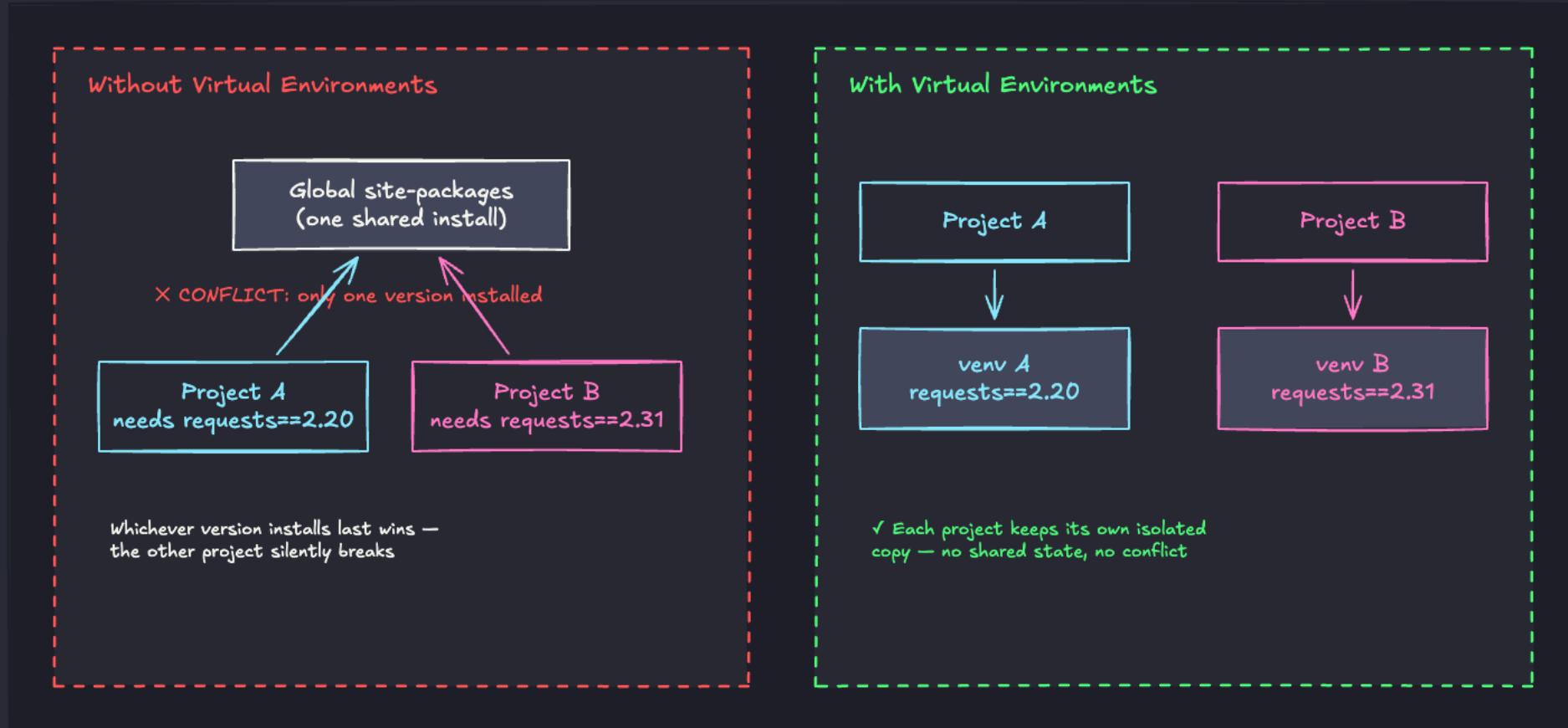
Python Virtual Environments



The Problem: One Python, Many Projects

- Your machine has **one** system Python install by default
- Every project you `pip install` into shares that same package library
- Two projects rarely want the *same* version of the *same* package
- Symptom: code that worked last week now throws `ImportError` or version errors

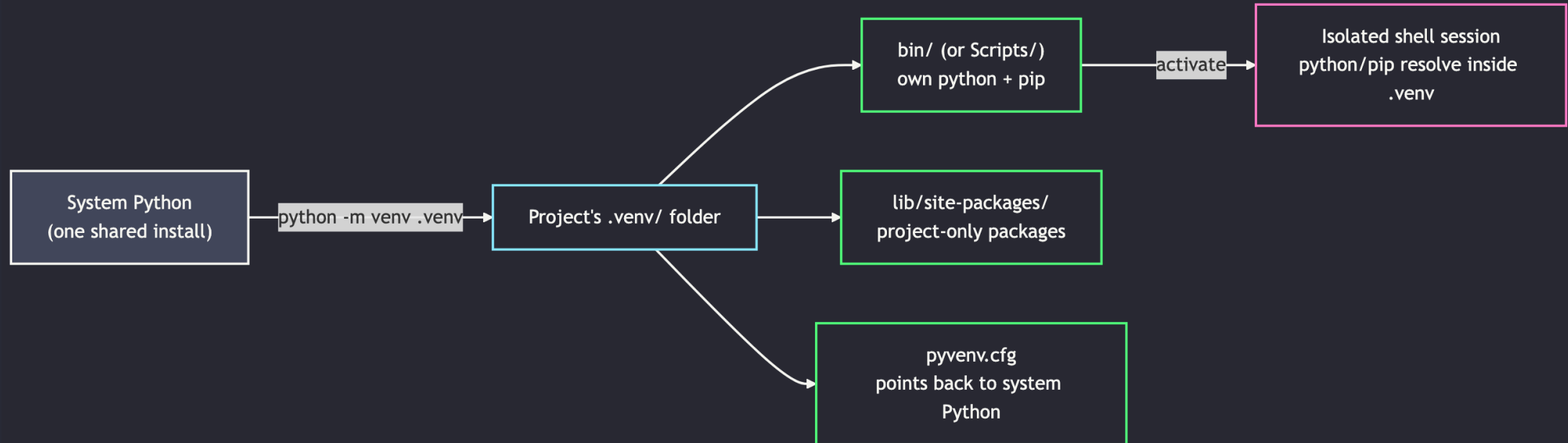
Without vs. With Virtual Environments



What a Virtual Environment Actually Is

- A **self-contained folder** holding its own Python interpreter and package library
- Not a separate Python install — it's a lightweight pointer back to your system Python
- Activating it changes *only* what `python` and `pip` resolve to in your current shell
- Deleting the folder removes the environment entirely — nothing else on your machine is affected

Anatomy of a `venv`



Tooling Landscape

- **venv** — built into Python 3.3+, zero extra install, what we'll demo today
- **uv** — much faster, manages Python versions too; used across our own course repos
 - Look for **pyproject.toml** + **uv.lock** in a project — that's the **uv** signature
- **conda** — popular in data science, manages non-Python dependencies too (e.g. C libraries)
- **poetry** — adds dependency resolution and packaging on top of venvs
- Today: stick with **venv** — every tool above builds on the same isolation concept

Demo: Create and Activate

```
# 1. Create a venv in your project folder
python -m venv .venv

# 2. Activate it
source .venv/bin/activate          # macOS / Linux
.venv\Scripts\activate             # Windows (PowerShell/cmd)

# 3. Confirm it's active – prompt shows (.venv), and:
which python                       # macOS / Linux
where python                       # Windows
```

Demo: Install, Freeze, Deactivate

```
# 4. Install a package – lands inside .venv, not globally
pip install requests

# 5. Snapshot exact versions for reproducibility
pip freeze > requirements.txt

# 6. Leave the venv
deactivate
```


Demo: Reproduce an Environment

```
# On a new machine, or a fresh clone of the project:  
python -m venv .venv  
source .venv/bin/activate  
pip install -r requirements.txt
```

- Anyone can recreate the *exact* same environment from `requirements.txt`
- This is what makes a project's setup reproducible for grading, collaboration, or deployment

Common Pitfalls

- **Forgetting to activate** before `pip install` — package lands in the global environment instead
- **Committing `.venv/` to git** — add it to `.gitignore`; commit `requirements.txt` instead
- **One venv per project** — don't reuse a single venv across unrelated projects
- **A broken venv is disposable** — just `deactivate`, delete the folder, and recreate it

Cheat Sheet

Action	Command
Create	<code>python -m venv .venv</code>
Activate (macOS/Linux)	<code>source .venv/bin/activate</code>
Activate (Windows)	<code>.venv\Scripts\activate</code>
Install a package	<code>pip install <package></code>
Save exact versions	<code>pip freeze > requirements.txt</code>
Reinstall from file	<code>pip install -r requirements.txt</code>
Deactivate	<code>deactivate</code>
Delete	remove the <code>.venv/</code> folder

References

Topic	Source
venv docs	Python Software Foundation. (2026). venv — Creation of virtual environments . Python 3 Documentation.
PEP 405	Eby, C. (2012). PEP 405 – Python Virtual Environments . Python Enhancement Proposals.
pip guide	Python Packaging Authority. (2026). Installing packages using pip and virtual environments . Python Packaging User Guide.
uv docs	Astral. (2026). uv — An extremely fast Python package manager . Astral Docs.

Recap & Practice

- A venv is a disposable, isolated copy of your project's packages
- Create → activate → install → freeze → deactivate
- **Before you write a single line of code on your next assignment: create a venv first**